PREMIUM EDITION

.NET & C# INTERVIEW ROADMAP

FRESHER TO MID-LEVEL · 75+ MODERN, TRICKY & MOST-ASKED QUESTIONS

PREPARED BY

Rutik Pimpale

C# 12 · .NET 8 · ASP.NET Core · EF Core · Async · LINQ · Web API

NAVIGATION

Table of Contents

CLICK ANY ENTRY TO JUMP TO THE SECTION

05 CATEGORY 5: ASP.NET CORE & .NET CORE

- Q48: .NET Framework vs .NET Core vs .NET 5+ ★
- Q49: Dependency Injection — Transient, Scoped, Singleton ★
- Q50: Middleware Pipeline in ASP.NET Core ★
- Q51: Filters — Action, Result, Exception, Authorization
- Q52: Garbage Collection — Generations 0, 1, 2
- Q53: IHttpClientFactory — Why Not new HttpClient()?
- Q54: IOptions Pattern — Strongly Typed Config
- Q55: appsettings.json and Configuration Hierarchy
- Q56: Hosted Services and BackgroundService
- Q57: Logging with ILogger
- Q58: Minimal APIs vs. Controllers (MODERN)
- Q59: Authentication vs. Authorization ★
- Q60: JWT Tokens in ASP.NET Core (MODERN)

06 CATEGORY 6: ENTITY FRAMEWORK CORE

- Q61: EF Core vs. ADO.NET vs. Dapper
- Q62: Code First vs. Database First ★
- Q63: EF Core Migrations
- Q64: AsNoTracking vs. Tracking (TRICKY)
- Q65: Lazy vs. Eager vs. Explicit Loading
- Q66: Include vs. ThenInclude
- Q67: Transactions in EF Core
- Q68: Stored Procedures with EF Core

07 CATEGORY 7: MODERN .NET & ADVANCED CONCEPTS

- Q69: Reflection — What It Is and Why It's Slow
- Q70: Delegates vs. Events
- Q71: Func vs. Action vs. Predicate (TRICKY)
- Q72: Custom Attributes
- Q73: Exception Handling — throw vs. throw ex (TRICKY)
- Q74: Records vs. Class vs. Struct (MODERN)
- Q75: Top-Level Statements (MODERN)
- Q76: Global Usings and File-Scoped Namespaces (MODERN)
- Q77: Span and Memory (MODERN)
- Q78: Source Generators (MODERN)

08 APPENDIX

- Quick Revision Cheat Sheet
- Interview Tips

CATEGORY 1: C# CORE FUNDAMENTALS

Q1: Value Types vs. Reference Types — Stack vs. Heap (IMPORTANT)

Direct Answer: Value types like `int`, `bool`, `struct`, `enum`, `decimal`, and `DateTime` store the actual data inline. Reference types like `class`, `string`, arrays, and delegates store a pointer to the data on the heap. The common phrase "value types live on the stack" is only partially true — a value type field inside a class actually lives on the heap with that class. So the real rule is: value types store data directly, reference types store an address.

- Value types: `int`, `bool`, `struct`, `enum`, `DateTime`.
- Reference types: `class`, `string`, `array`, `interface`, `delegate`.
- Where the variable lives depends on where it's declared.
- Reference types can be `null`; value types cannot (unless nullable).

```
int x = 5;                      // value type, stack
Order o = new Order { Id = 99 }; // reference, object on heap
```

Q: Is `string` a value or reference type? **A:** Reference type, but it behaves like a value because it's immutable.

Q: Where does an `int` field inside a class live? **A:** On the heap — because the class itself lives on the heap.

Q: Can a value type be `null`? **A:** Only if it's declared as nullable (`int?`).

Q2: Boxing and Unboxing — Performance Cost (IMPORTANT)

Direct Answer: Boxing means converting a value type into an `object` reference, which allocates memory on the heap. Unboxing means casting that object back to its original value type. Each boxing operation costs a heap allocation, a copy, and adds GC pressure — roughly 20–30× slower than working with the value directly. This is why generics like `List<int>` were introduced — they completely avoid boxing.

- Boxing → value type to `object` (heap allocation).
- Unboxing → casting back from object.
- Slows down performance and increases GC.

- Avoid using non-generic collections like `ArrayList`.

```
int i = 10;
object obj = i;       // boxing
int j = (int)obj;     // unboxing
```

Q: How do you avoid boxing? **A:** Use generic collections like `List<T>`, `Dictionary<K,V>`.

Q: Does `int.ToString()` cause boxing? **A:** No, because `int` has its own override for `ToString()`.

Q: Why is boxing bad for performance? **A:** It allocates on the heap, copies data, and adds GC overhead.

Q3: struct vs. class — When to Choose a struct?

Direct Answer: Use a `struct` for small, immutable, value-like types such as `Point`, `Money`, or coordinates. Use a `class` when you need identity, inheritance, polymorphism, or shared mutable state. Structs are value types (copied on assignment), classes are reference types (shared via reference). A common guideline is: if your type is under ~16 bytes and immutable, use a struct.

- Struct = value type, copied by value, no inheritance.
- Class = reference type, supports inheritance, shared by reference.
- Mutable structs are dangerous due to copying.
- Structs cannot be `null` unless made nullable.

```
public readonly struct Point { public int X { get; } public int Y { get; } }
public class Customer { public string Name { get; set; } }
```

Q: Can a struct have a constructor? **A:** Yes, but it must initialize all fields.

Q: Can struct inherit another struct? **A:** No, structs only support implementing interfaces.

Q: Why are mutable structs risky? **A:** Because every assignment creates a copy, and modifications on copies are lost.

Q4: const vs. readonly vs. static readonly (TRICKY)

Direct Answer: `const` is a compile-time constant — its value is baked into the compiled code and must be set at declaration. `readonly` is set at runtime and can only be assigned in declaration or constructor; values can differ per instance. `static readonly` is similar to `readonly` but shared across all instances and resolved at runtime — the safe choice for cross-assembly constants.

- `const` → compile-time, only literals (int, string, etc.).
- `readonly` → runtime, set in constructor.
- `static readonly` → runtime, shared across instances.
- Use `static readonly` for cross-assembly safety.

```
public const double Pi = 3.14;
public static readonly DateTime AppStart = DateTime.Now;
public readonly int Id;    // set in ctor
```

Q: Why prefer `static readonly` over `const` in libraries? **A:** `const` values are baked into consumer assemblies — changes won't reflect without rebuild.

Q: Can `const` be a `DateTime`? **A:** No, only primitive literals and strings.

Q: Can `readonly` be modified? **A:** Only inside the constructor or at declaration.

Q5: Why Are Strings Immutable in C#?

Direct Answer: Strings in C# are immutable, meaning once a string is created, its value cannot be changed. Any "modification" actually creates a new string and the old one becomes garbage. Immutability gives thread safety, safe hashing for dictionary keys, and predictable behavior. The downside is that `+=` in a loop creates many intermediate strings, which is slow — that's where `StringBuilder` helps.

- Strings cannot be changed after creation.
- Each modification creates a new object.
- Provides thread safety automatically.
- Use `StringBuilder` for heavy concatenations.

```
string s = "Hello";
s += " World";    // creates a new string, original is GC'd
```

Q: Why is string a reference type but acts like a value? **A:** Because of immutability — comparing two strings checks values, not references.

Q: Are two equal strings stored only once? **A:** Yes, due to string interning of literals at compile time.

Q: Can we make our own immutable type? **A:** Yes, use `readonly` fields and only `get` properties.

Q6: String vs. StringBuilder — When to Use Which?

Direct Answer: Use `StringBuilder` when you're building a string in a loop or doing many concatenations. It uses an internal mutable buffer, so it doesn't keep allocating new string objects. For 2–3 concatenations, plain `+` or string interpolation is faster and cleaner. As a thumb rule: more than 5–10 concatenations or any loop → `StringBuilder`.

- String → immutable, slow for many edits.
- StringBuilder → mutable buffer, much faster in loops.
- Pre-set capacity if size is known.
- For small joins, prefer `string.Join` or interpolation.

```
var sb = new StringBuilder();
foreach (var item in items)
    sb.Append(item).Append(",");
return sb.ToString();
```

Q: Is `StringBuilder` thread-safe? **A:** No, you should not share a single instance across threads.

Q: When should I avoid `StringBuilder`? **A:** For 2–3 small concatenations — `string.Concat` or interpolation is faster.

Q: Does `string.Join` use a `StringBuilder`? **A:** Internally it uses something similar, optimized for known lengths.

Q7: var vs. dynamic vs. object — Common Confusion (TRICKY)

Direct Answer: `var` is implicit typing — the compiler figures out the actual type at compile time, so it's still strongly typed. `dynamic` skips compile-time type checking and resolves everything at

runtime. `object` is the base class of all types — you must cast it to use it. So `var` is safe and fast, `dynamic` is flexible but risky, and `object` is general but requires casting.

- `var` → compile-time type, strongly typed.
- `dynamic` → runtime type, no IntelliSense.
- `object` → base type, needs casting.
- Use `var` for clean code, `dynamic` for COM/JSON only.

```
var x = "Hello";        // string
dynamic d = "Hello";    // resolved at runtime
object o = "Hello";     // requires cast: (string)o
```

Q: Is `var` the same as JavaScript's `var`? **A:** No, C#'s `var` is fully strongly typed at compile time.

Q: When to use `dynamic`? **A:** For COM interop, ExpandoObject, or dynamic JSON.

Q: Can `var` be used as a class field? **A:** No, only inside methods with an initializer.

Q8: Nullable Types and the ?? Operator

Direct Answer: `int?` (shorthand for `Nullable<int>`) lets a value type also represent the absence of a value. The null-coalescing operator `??` returns the right-hand value if the left is `null`. The `??=` operator assigns only when the variable is null. Modern C# also has nullable reference types (`string?`) which are compiler-checked annotations to catch null reference exceptions early.

- `int?` allows value types to be null.
- `??` returns alternative value when null.
- `??=` assigns if null.
- `?.` (null-conditional) prevents null-deref crashes.

```
int? age = null;
int safe = age ?? 0;        // 0 if null
age ??= 18;                 // assigns 18 if null
string? name = user?.Name; // null-safe access
```

Q: Difference between `int` and `int?`? **A:** `int?` can hold null; `int` cannot.

Q: What does `?.` operator do? **A:** Returns null if the operand is null, avoiding NullReferenceException.

Q: How do you check if nullable has a value? **A:** Using `.HasValue` or `≠ null`.

Q9: ref vs. out vs. in Parameters

Direct Answer: `ref` passes a variable by reference — the caller must initialize it, and the method can read or modify it. `out` is also pass-by-reference but the method **must** assign a value before returning, and the caller doesn't need to initialize. `in` is pass-by-reference for read-only access — used to avoid copying large structs without allowing modification.

- `ref` → caller initializes, method reads/writes.
- `out` → method must assign, caller needn't initialize.
- `in` → read-only reference, performance optimization.
- C# 7+ tuple returns often replace `out`.

```
public bool TryParse(string s, out int value) {
    return int.TryParse(s, out value);
}
public void Increment(ref int counter) ⇒ counter++;
public bool Contains(in Rectangle r, Point p) { /* read-only */ }
```

Q: Modern alternative to `out`? **A:** Tuple return types like `(bool ok, int value)`.

Q: Why use `in`? **A:** Avoids copying large structs while keeping them read-only.

Q: Can we overload methods using `ref`/`out`? **A:** Yes, but `ref` and `out` together can't differ for overloading.

Q10: The using Statement and IDisposable

Direct Answer: The `using` statement automatically calls `.Dispose()` on an object when the block ends, even if an exception is thrown. Internally, it compiles to a `try/finally` block. This is essential for unmanaged resources like database connections, file streams, and HTTP clients. C# 8 introduced `using var` (without braces) which disposes when the enclosing method scope ends.

- Ensures `Dispose()` is always called.
- Prevents resource leaks (DB, file, sockets).
- `using var` is shorter, modern syntax.
- `await using` works with `IAsyncDisposable`.

```
using var conn = new SqlConnection(connectionString);
conn.Open();
// auto-disposed at method end
```

Q: What happens without `using`? **A:** Resources may leak; e.g., DB connections fill the pool.

Q: Difference: `using` statement vs. directive? **A:** Directive imports namespaces; statement disposes resources.

Q: What is `IAsyncDisposable`? **A:** Async cleanup interface used with `await using` (e.g., DbContext).

Q11: Pattern Matching in C# (MODERN)

Direct Answer: Pattern matching lets you check a value's shape and extract data in one expression. Modern C# supports type patterns, property patterns, tuple patterns, and `switch` expressions. It makes code more readable and safer than long `if-else` or `switch-case` chains. Introduced in C# 7 and significantly enhanced through C# 8, 9, 10, and 11.

- `is` keyword for type checks with assignment.
- `switch` expressions return values.
- Property and tuple patterns extract data.
- Cleaner than nested `if/else`.

```
object obj = "hello";
if (obj is string s) Console.WriteLine(s.Length);

string Describe(int n) => n switch {
    < 0 => "Negative",
    0   => "Zero",
    > 0 => "Positive"
};
```

Q: What is a switch expression? **A:** A concise version of `switch` that returns a value.

Q: What is a property pattern? **A:** Matches based on object property values, e.g., `{ Status: "Active" }`.

Q: Difference between `is` and `as`? **A:** `is` returns bool; `as` returns the cast object or null.

Q12: Init-Only Properties and Required Members (MODERN)

Direct Answer: `init` (C# 9) lets a property be set only during object initialization, making it immutable afterward. The `required` modifier (C# 11) forces callers to set that property when creating the object. Together they create safer immutable types without writing constructors. They're commonly used in DTOs, records, and configuration classes.

- `init` → settable only during construction.
- `required` → must be set by the caller.
- Great for immutability and DTOs.
- Replaces some constructor boilerplate.

```
public class User {
    public required string Name { get; init; }
    public int Age { get; init; }
}
var u = new User { Name = "Rutik", Age = 25 };
```

Q: Difference between `init` and `set`? **A:** `set` allows future changes; `init` only at object creation.

Q: What if `required` property is missing? **A:** Compiler error — caller must set it.

Q: Where are these used most? **A:** DTOs, records, immutable models, options classes.

CATEGORY 2: OBJECT-ORIENTED PROGRAMMING

Q13: The 4 Pillars of OOP (IMPORTANT)

Direct Answer: The four pillars are **Encapsulation, Abstraction, Inheritance, and Polymorphism**. Encapsulation hides internal state behind methods/properties. Abstraction shows only the necessary functionality and hides implementation details. Inheritance lets one class reuse another's behavior. Polymorphism lets a single interface represent different underlying types — through method overriding or interfaces.

- Encapsulation → data hiding via properties.
- Abstraction → expose what, hide how.
- Inheritance → "is-a" relationship.

- Polymorphism → one interface, many forms.

```
public abstract class Shape {           // abstraction
    public abstract double Area();      // polymorphism
}
public class Circle : Shape {           // inheritance
    private double _r;                  // encapsulation
    public Circle(double r) ⇒ _r = r;
    public override double Area() ⇒ Math.PI * _r * _r;
}
```

Q: Difference between abstraction and encapsulation? **A:** Abstraction hides complexity; encapsulation hides data.

Q: Real-life example of polymorphism? **A:** A `Notification` interface used by Email, SMS, Push notifications.

Q: Does C# support multiple inheritance? **A:** Not for classes — only via multiple interfaces.

Q14: Interface vs. Abstract Class — When to Use What? (IMPORTANT)

Direct Answer: Use an **interface** to define a pure contract — a "can-do" capability that any class can implement. Use an **abstract class** when you have shared state and partial implementation that derived classes should extend. A class can implement many interfaces but inherit from only one abstract class. From C# 8 onwards, interfaces can also have default implementations for non-breaking API evolution.

- Interface → contract, no fields.
- Abstract class → partial implementation, shared logic.
- Multiple interfaces, single abstract base.
- C# 8+ allows default interface methods.

```
public interface IExportable { void Export(); }
public abstract class ReportBase {
    public abstract string Title { get; }
    public virtual string Footer() ⇒ "Generated today";
}
```

Q: Can interfaces have fields? **A:** No, only properties, methods, and events.

Q: Can abstract class have a constructor? **A:** Yes, called by derived classes via `base()`.

Q: Can interface have private methods? **A:** Yes, since C# 8 (only with default implementation).

Q15: Method Overloading vs. Method Overriding

Direct Answer: Overloading means having multiple methods with the **same name but different parameters** in the same class — resolved at compile time (static polymorphism). Overriding means a derived class **replaces** a base class's `virtual` or `abstract` method using the `override` keyword — resolved at runtime (dynamic polymorphism). Overloading deals with signature differences, overriding deals with inheritance.

- Overloading → same name, different parameters.
- Overriding → child replaces parent's virtual/abstract method.
- Overloading is compile-time, overriding is runtime.
- Return type alone cannot differentiate overloads.

```
class Calc {
    public int Add(int a, int b) => a + b;          // overload
    public double Add(double a, double b) => a + b; // overload
}
class Animal { public virtual void Speak() => Console.WriteLine("..."); }
class Dog : Animal { public override void Speak() => Console.WriteLine("Bark"); }
```

Q: What is method hiding? **A:** Using `new` to hide a parent's method instead of overriding.

Q: Can return type alone differ for overloads? **A:** No, parameter list must differ.

Q: Why is `override` better than `new`? **A:** It uses runtime polymorphism — works through base reference.

Q16: virtual vs. abstract vs. sealed (TRICKY)

Direct Answer: `virtual` allows a method to be overridden in derived classes but provides a default implementation. `abstract` requires derived classes to provide an implementation and exists only in abstract classes. `sealed` prevents further inheritance (on a class) or further overriding (on a method). Use `virtual` for optional extension, `abstract` for mandatory contracts, and `sealed` for stopping inheritance.

- `virtual` → can be overridden, has default body.
- `abstract` → must be overridden, no body.
- `sealed` → cannot be inherited or overridden further.
- `sealed override` ends overriding chain.

```
public abstract class Shape {
    public abstract double Area();              // mandatory
    public virtual string Describe() => "Shape"; // optional
}
public sealed class Square : Shape {            // cannot inherit
    public override double Area() => 25;
}
```

Q: Why use `sealed`? **A:** For performance (JIT can devirtualize) and design clarity.

Q: Can a sealed class implement interfaces? **A:** Yes, sealed only prevents inheritance.

Q: Why is `string` sealed? **A:** For safety, immutability, and performance.

Q17: Constructor Chaining — `this()` and `base()`

Direct Answer: Constructor chaining means one constructor calls another to avoid duplicate initialization. Use `this(...)` to call another constructor in the same class, and `base(...)` to call the parent class's constructor. They must appear before the constructor body. C# 12 also added **primary constructors** for cleaner initialization syntax.

- `this()` → same class constructor.
- `base()` → parent class constructor.
- Avoids duplicate initialization code.
- C# 12 supports primary constructors.

```
public class Person {
    public Person() : this("Unknown") { }
    public Person(string name) { Name = name; }
    public string Name { get; }
}

// C# 12 primary constructor
public class Logger(string source) {
    public string Source { get; } = source;
}
```

Q: Can constructor be private? **A:** Yes — used in singletons and factory methods.

Q: Are constructors inherited? **A:** No, but they can be called via `base(...)`.

Q: Can we call virtual methods from constructor? **A:** Possible but risky — derived fields may not be initialized yet.

Q18: Static vs. Instance Members

Direct Answer: Static members belong to the class itself, not to any instance — they're shared across all objects and accessed using the class name. Instance members belong to a specific object and need an instance to be accessed. Static is great for utilities (`Math.Sqrt`), constants, and shared state. Avoid mutable static state in multi-threaded apps because it can cause race conditions.

- Static → one copy per class, no instance needed.
- Instance → one copy per object.
- Static methods can't access instance members.
- Static is shared across threads — handle locking.

```
public class Counter {
    public static int Total;       // shared
    public int Local;              // per instance
}
Counter.Total++;                   // accessed via class
```

Q: Can static class be instantiated? **A:** No, it's sealed by default.

Q: Why are extension methods static? **A:** Because they belong to a static class.

Q: Are static fields thread-safe? **A:** No, you need locks or `Interlocked`.

Q19: Composition over Inheritance

Direct Answer: Composition means building a class by **holding references** to other objects, while inheritance extends from a base class. Composition is preferred because it's more flexible, easier to test, and avoids tight coupling. Inheritance is rigid — once you commit to a hierarchy, changes ripple downward. Modern .NET, especially DI in ASP.NET Core, is built around composition.

- Composition → "has-a" relationship.
- Inheritance → "is-a" relationship.
- Composition is more flexible and testable.
- DI is real-world composition.

```
public class OrderService {
    private readonly INotifier _notifier;
    public OrderService(INotifier n) => _notifier = n;
    public void Place(Order o) => _notifier.Send($"Placed {o.Id}");
}
```

Q: When is inheritance still useful? **A:** When there's a clear "is-a" relationship and shared logic.

Q: Is dependency injection composition? **A:** Yes — services receive dependencies instead of inheriting them.

Q: Drawback of inheritance? **A:** Tight coupling and the fragile base class problem.

Q20: Deep Copy vs. Shallow Copy

Direct Answer: A **shallow copy** creates a new object but copies references for nested objects, so they're shared between copies. A **deep copy** creates a fully independent copy including all nested objects. `MemberwiseClone()` in .NET produces a shallow copy. For deep copies, you write your own copy logic, use serialization, or use records with `with` expressions.

- Shallow → top-level copy, nested objects shared.
- Deep → fully independent copy.
- `MemberwiseClone()` = shallow.
- Records support `with` expressions for cloning.

```
public record OrderDto(int Id, string Status);
var original = new OrderDto(1, "Pending");
var shipped = original with { Status = "Shipped" };
```

Q: Is JSON serialization a deep copy? **A:** Yes, but slower and may break on circular references.

Q: Is `MemberwiseClone()` deep or shallow? **A:** Shallow — shares nested references.

Q: Use case for deep copy? **A:** Snapshotting state, undo/redo, isolating test data.

Q21: SOLID Principles (IMPORTANT)

SOLID is five design principles that make code maintainable and testable:

- **S** — Single Responsibility: one class, one reason to change.

- **O** — Open/Closed: open to extension, closed to modification.
- **L** — Liskov Substitution: subclasses must work where parent works.
- **I** — Interface Segregation: small focused interfaces over fat ones.
- **D** — Dependency Inversion: depend on abstractions, not concretions.

- Reduces coupling and increases testability.
- ASP.NET Core's DI follows DIP.
- Avoid "God classes" doing too much.
- Apply where change is likely; don't over-engineer.

```
// Following DIP and SRP
public class OrderService {
    public OrderService(IOrderRepository repo, IEmailSender mail) { }
}
```

Q: Most commonly applied SOLID principle? **A:** SRP and DIP — they reduce coupling.

Q: Real example of OCP? **A:** Adding new payment types via interfaces without modifying existing code.

Q: Does DI implement DIP? **A:** Yes, DI containers are the practical form of DIP.

Q22: Encapsulation in the Real World

Direct Answer: Encapsulation means hiding the internal state of an object and exposing it only through controlled methods or properties. In C#, we use access modifiers (`private`, `protected`, etc.) and properties with getters/setters to control how data is read or modified. This protects invariants — for example, an Account class wouldn't allow setting balance directly; only `Deposit` and `Withdraw` modify it.

- Hide fields, expose via properties.
- Use access modifiers wisely.
- Maintain invariants (validation in setter).
- Protects against misuse.

```
public class Account {
    private decimal _balance;
    public decimal Balance => _balance;
    public void Deposit(decimal amt) {
        if (amt <= 0) throw new ArgumentException();
        _balance += amt;
    }
}
```

Q: Why use properties over public fields? **A:** They allow validation, lazy loading, and binding.

Q: What is auto-property? **A:** A shorthand: `public int Id { get; set; }` — compiler creates the field.

Q: Can encapsulation hide method behavior too? **A:** Yes, via abstraction — exposing only what's needed.

Q23: The base Keyword

Direct Answer: The `base` keyword lets a derived class access methods or constructors of its direct parent class. It's mostly used when overriding a method to extend (not replace) the parent's logic, or in constructors to call the base constructor. It's resolved at compile time and refers strictly to the immediate parent — you can't skip levels.

- `base.Method()` → calls parent's version.
- `base()` → calls parent constructor.
- Used to extend rather than replace.
- Cannot use `base` in static methods.

```
public class AuditedRepo : Repository {
    public override void Save(Entity e) {
        Log(e);
        base.Save(e);   // delegate to parent
    }
}
```

Q: Can `base` skip multiple levels? **A:** No, only direct parent.

Q: Can constructor use both `this` and `base`? **A:** Only one of them at a time.

Q: What if you forget `base.Dispose()` in a child? **A:** Resources of the base class may leak.

Q24: Multiple Inheritance in C# (TRICKY)

Direct Answer: C# does **not support multiple inheritance** for classes — a class can inherit from only one base class. This avoids the "diamond problem" of ambiguous method resolution. However, a class can implement **multiple interfaces**, achieving similar flexibility. Since C# 8, interfaces can also have default method implementations, providing a controlled form of multiple inheritance.

- Single class inheritance only.
- Multiple interface implementation allowed.
- Avoids diamond problem ambiguity.
- Default interface methods (C# 8+) bring partial multi-inheritance.

```
public interface IReader { void Read(); }
public interface IWriter { void Write(); }
public class FileHandler : IReader, IWriter {
    public void Read() { }
    public void Write() { }
}
```

Q: Why doesn't C# allow multiple class inheritance? **A:** To prevent ambiguity in method resolution (diamond problem).

Q: What if two interfaces have same default method? **A:** The class must explicitly implement and disambiguate.

Q: Can struct inherit from class? **A:** No, structs only implement interfaces.

CATEGORY 3: COLLECTIONS & LINQ

Q25: Array vs. ArrayList vs. List

Direct Answer: `Array` has a fixed size and is type-safe. `ArrayList` is older, dynamic, but stores everything as `object` — meaning value types get boxed and there's no type safety. `List<T>` is the modern, generic, type-safe, and dynamic choice. In real projects, we almost always use `List<T>` and rarely touch `ArrayList`, which is a leftover from .NET 1.0.

- Array → fixed size, type-safe.
- ArrayList → dynamic but old, boxes values.

- List → generic, dynamic, fast.
- Use `List<T>` by default.

```
List<int> numbers = new() { 1, 2, 3 };
int[] arr = new int[5];
```

Q: Why avoid `ArrayList`? **A:** It causes boxing for value types and lacks type safety.

Q: Is `List<T>` thread-safe? **A:** No, use `ConcurrentBag<T>` or locks for threading.

Q: How does `List<T>` grow? **A:** It doubles internal capacity when full.

Q26: IEnumerable vs. IQueryable — Which Filters at the DB? (IMPORTANT)

Direct Answer: `IQueryable` builds an expression tree that EF Core translates into SQL — so filtering happens **at the database**. `IEnumerable` represents in-memory iteration — filtering happens **in your application** after data is loaded. Calling `.ToList()` too early forces data into memory and you lose database-level filtering, which can crash apps when tables are large.

- IQueryable → filters at the database (SQL).
- IEnumerable → filters in memory.
- Avoid premature `.ToList()`.
- IQueryable supports expression composition.

```
var pending = ctx.Orders
    .Where(o => o.Status == "Pending")  // SQL WHERE
    .ToListAsync();
```

Q: How to convert IQueryable to IEnumerable? **A:** Call `.AsEnumerable()` or `.ToList()`.

Q: Why is mixing them dangerous? **A:** Filtering may move from SQL to memory and pull entire tables.

Q: Is IQueryable always faster? **A:** Usually — but only when filtering reduces data significantly.

Q27: Deferred Execution in LINQ

Direct Answer: LINQ queries don't execute when defined — they only execute when iterated. Operators like `Where`, `Select`, and `OrderBy` build a query description; execution happens on

`foreach`, `ToList`, `Count`, `First`, etc. This allows efficient query composition but can cause unexpected re-execution if the same query is iterated multiple times.

- Queries are lazy by default.
- Each iteration re-runs the query.
- Use `.ToList()` to materialize once.
- Combines well with `IQueryable` for SQL.

```
var query = list.Where(x => x > 10);  // not executed yet
foreach (var x in query) { }          // executes here
```

Q: What triggers immediate execution? **A:** `ToList`, `ToArray`, `Count`, `First`, `Sum`.

Q: Why is deferred execution useful? **A:** Lets you build queries piece by piece without running prematurely.

Q: What's a common bug with deferred execution? **A:** Captured variable changes affecting later iteration results.

Q28: How Does Dictionary Handle Collisions?

Direct Answer: Dictionary in .NET uses a hash table with **separate chaining**. Each key's hash code maps to a bucket; if multiple keys hash to the same bucket, they form a linked chain. On lookup, the dictionary walks that chain comparing keys with `Equals`. Average lookup is O(1) but can degrade to O(n) if all keys collide. Stable `GetHashCode` and `Equals` are essential.

- Hash table with chaining.
- Average O(1), worst O(n) lookup.
- Don't use mutable objects as keys.
- Use `ConcurrentDictionary` for thread safety.

```
var dict = new Dictionary<string, int>();
dict["age"] = 25;
dict.TryGetValue("age", out int v);
```

Q: What if a key already exists? **A:** `Add` throws; indexer `[]` overwrites.

Q: Why must keys be immutable? **A:** Changing a key's hash makes the entry unreachable.

Q: When to use `ConcurrentDictionary`? **A:** When multiple threads read/write the same dictionary.

Q29: yield return — Custom Lazy Iteration

Direct Answer: `yield return` produces values one at a time on demand instead of building the full collection in memory. The compiler generates a state machine that pauses execution after each yield and resumes on the next iteration. It's perfect for streaming large datasets, paged APIs, or infinite sequences without consuming memory.

- Produces values lazily.
- Saves memory for large data.
- Pauses and resumes execution automatically.
- Used in custom iterators and LINQ-style methods.

```
IEnumerable<int> GetEvens() {
    for (int i = 0; i < 100; i++)
        if (i % 2 == 0) yield return i;
}
```

Q: When does the method body actually run? **A:** When iteration starts (first `MoveNext`).

Q: Async version of yield? **A:** Use `IAsyncEnumerable<T>` with async + `yield return`.

Q: Can we use `yield` in an async method? **A:** Only with `IAsyncEnumerable`, not regular Task.

Q30: Any() vs. Count() > 0 — Which Is Faster? (TRICKY)

Direct Answer: `Any()` is faster on `IEnumerable<T>` because it stops at the first matching element — short-circuits. `Count() > 0` walks the entire collection. But for `List<T>` or arrays, `.Count` is O(1) since it's a property, slightly beating `Any()`. For `IQueryable<T>` (EF Core), `Any()` translates to SQL `EXISTS`, much faster than `COUNT(*)`.

- IEnumerable → use `Any()` (short-circuits).
- List/Array → use `.Count > 0` (O(1)).
- IQueryable → use `Any()` (SQL EXISTS).
- Avoid `Count() > 0` in big sequences.

```
if (orders.Any()) { ... }
if (await ctx.Orders.AnyAsync()) { ... }   // SQL EXISTS
```

Q: Why is `Count()` slow on IEnumerable? **A:** It iterates the full sequence even if not needed.

Q: Does EF Core optimize `Any()`? **A:** Yes, generates SQL `EXISTS`.

Q: When can `Count()` be O(1)? **A:** On types implementing `ICollection<T>` like `List<T>`.

Q31: Select vs. SelectMany

Direct Answer: `Select` projects each element into a new shape — one to one mapping. `SelectMany` projects each element into a **collection** and then flattens all those collections into a single sequence — one to many mapping. Use `Select` for transformations and `SelectMany` when you have nested collections you want to merge.

- Select → 1 to 1 mapping.
- SelectMany → 1 to many, flattens.
- Acts like a SQL JOIN in EF Core.
- Common with parent-child data.

```
var allItems = orders.SelectMany(o ⇒ o.Items);          // flat
var listOfLists = orders.Select(o ⇒ o.Items);           // nested
```

Q: Output type difference? **A:** Select → `IEnumerable<IEnumerable<T>>`; SelectMany → `IEnumerable<T>`.

Q: Real-world use of SelectMany? **A:** Getting all line items across all orders.

Q: Does it generate SQL JOIN in EF Core? **A:** Yes, typically.

Q32: First vs. FirstOrDefault vs. Single vs. SingleOrDefault (TRICKY)

Direct Answer: `First` returns the first matching element and **throws** if none found. `FirstOrDefault` returns the first match or default (null/zero) if none. `Single` ensures **exactly one** match, throws if zero or more than one. `SingleOrDefault` allows zero or one match — throws if more than one. Use `Single` when uniqueness must be guaranteed.

- `First` → first element, throws if empty.
- `FirstOrDefault` → first or default value.
- `Single` → exactly one, throws otherwise.
- `SingleOrDefault` → zero or one, throws if more.

```
var u1 = users.First(u => u.Active);                    // may throw
var u2 = users.FirstOrDefault(u => u.Active);           // null if not found
var u3 = users.Single(u => u.Email == "x@y.com");       // exactly one
```

Q: Which is safest? **A:** `FirstOrDefault` — handles missing data gracefully.

Q: When to use `Single`? **A:** When data uniqueness must be enforced (e.g., primary key).

Q: Performance difference? **A:** `Single` always scans for second match; `First` stops at first.

Q33: GroupBy in LINQ

Direct Answer: `GroupBy` groups elements by a key and returns an `IGrouping<TKey, TElement>` for each group. Each group exposes the key and the elements. It's commonly used for aggregations like totals per category or counts per status. In EF Core, simple `GroupBy` translates to SQL `GROUP BY`, but complex grouping may run client-side.

- Groups items by key.
- Returns `IGrouping<TKey, TElement>`.
- Common with aggregation (Sum, Count, Avg).
- EF Core translates simple cases to SQL.

```
var totals = orders
    .GroupBy(o => o.CustomerId)
    .Select(g => new { Customer = g.Key, Total = g.Sum(o => o.Amount) });
```

Q: What does `IGrouping` contain? **A:** Key and the matching elements as IEnumerable.

Q: Does GroupBy execute SQL or in memory? **A:** Depends on EF Core — simple keys go to SQL.

Q: Use case in real apps? **A:** Sales report by region, error count by status, etc.

Q34: ICollection vs. IList vs. IReadOnlyList

Direct Answer: `ICollection<T>` supports basic operations like Add, Remove, Count — no indexing. `IList<T>` extends it with index access (`list[0]`) and ordering. `IReadOnlyList<T>` is for read-only consumption — it has indexing and Count but no modification. As a rule, return `IReadOnlyList<T>` from APIs to prevent callers from mutating internal state.

- ICollection → basic ops, no indexer.
- IList → indexed, mutable.
- IReadOnlyList → indexed, read-only.
- Return `IReadOnlyList<T>` to expose data safely.

```
public IReadOnlyList<Order> GetRecent() ⇒ _store.ToList();
```

Q: Difference between IList and IReadOnlyList? **A:** IList allows changes; IReadOnlyList does not.

Q: Does List implement both? **A:** Yes, it implements IList and IReadOnlyList.

Q: Why expose IReadOnly? **A:** Prevents accidental mutation by callers.

Q35: HashSet vs. List — When to Use HashSet (TRICKY)

Direct Answer: `List<T>` keeps insertion order and allows duplicates — best for sequential or indexed access. `HashSet<T>` stores unique items and provides O(1) lookup using hashing — best for membership checks. If you frequently check if an item exists or want to remove duplicates, use `HashSet<T>`. If you need ordering or duplicates, use `List<T>`.

- List → ordered, duplicates allowed, O(n) lookup.
- HashSet → unique items, O(1) lookup.
- HashSet has no indexer.
- Use HashSet for fast Contains checks.

```
var visited = new HashSet<string>();
if (!visited.Contains(id)) visited.Add(id);
```

Q: Can HashSet have null? **A:** Yes, but only one null entry.

Q: What's the order of HashSet? **A:** No guaranteed order.

Q: Best use case? **A:** Removing duplicates, fast lookup of unique IDs.

Q36: LINQ Method Syntax vs. Query Syntax

Direct Answer: LINQ supports two styles: **method syntax** uses extension methods (`.Where().Select()`), and **query syntax** uses SQL-like keywords

(`from x in list where ... select ...`). They're functionally identical and compile to the same code. Method syntax is more popular today because it's flexible, supports all operators, and chains cleanly. Query syntax is more readable for joins and group-bys.

- Method syntax → extension methods chain.
- Query syntax → SQL-like keywords.
- Identical results, equal performance.
- Method syntax supports more operators.

```
// method syntax
var result = orders.Where(o => o.Total > 100).Select(o => o.Id);
// query syntax
var result2 = from o in orders where o.Total > 100 select o.Id;
```

Q: Are they different in performance? **A:** No — query syntax compiles to method calls.

Q: Which supports more operators? **A:** Method syntax (e.g., `Distinct`, `Skip`, `Take`).

Q: When is query syntax preferred? **A:** For readable joins and group operations.

CATEGORY 4: ASYNC, MULTITHREADING & PARALLELISM

Q37: Process vs. Thread vs. Task (IMPORTANT)

Direct Answer: A **Process** is an OS-level running program with its own memory. A **Thread** is the smallest execution unit inside a process, sharing memory with sibling threads. A **Task** is a higher-level .NET abstraction for asynchronous work — usually scheduled on the thread pool. Tasks are lighter than threads and provide composability via `WhenAll`, `WhenAny`, and async/await.

- Process → isolated memory, heavy.
- Thread → shares memory, OS-level.
- Task → lightweight, uses thread pool.
- Use `Task` and async/await for modern apps.

```
Task.Run(() => Compute());          // background work
await Task.Delay(1000);             // non-blocking wait
```

Q: Difference between Task and Thread? **A:** Thread is OS-level; Task is a unit of work that runs on threads.

Q: Does Task always create a new thread? **A:** No, it uses the thread pool.

Q: What's the unit of CPU scheduling? **A:** A thread, not a task.

Q38: async and await — How They Work (IMPORTANT)

Direct Answer: `async` and `await` enable non-blocking asynchronous code. When you `await` an I/O operation like a DB or HTTP call, the current thread is released to do other work. When the operation completes, execution resumes — possibly on a different thread. Importantly, `async` doesn't create new threads; it just lets the runtime use them more efficiently. This is why ASP.NET Core scales so well.

- async/await → non-blocking I/O.
- Doesn't create new threads.
- Compiler builds a state machine internally.
- Improves scalability under load.

```
public async Task<User> GetUserAsync(int id) {
    return await _db.Users.FindAsync(id);
}
```

Q: Does async make code parallel? **A:** No, it makes code non-blocking.

Q: Should every method be async? **A:** Only methods doing I/O or awaiting tasks.

Q: What is `Task.CompletedTask`? **A:** A pre-completed task — useful in async methods with no work to do.

Q39: Task vs. Thread — When to Use Which?

Direct Answer: Use `Task` for almost everything — short-lived units of work, async I/O, parallel CPU operations. Tasks are scheduled on the .NET thread pool, which reuses threads efficiently. Use raw `Thread` only for long-running, dedicated workers (rare today). Modern code virtually never uses `new Thread()` directly; we use `Task.Run` for CPU work and `async`/`await` for I/O.

- Task → modern, lightweight, thread pool.
- Thread → heavy, OS-level.

- Use `Task.Run` for CPU-bound.
- Use `async/await` for I/O-bound.

```
await Task.Run(() => ComputeHash(file));    // CPU-bound
var data = await _http.GetStringAsync(url); // I/O-bound (no Task.Run!)
```

Q: When should I use raw Thread? **A:** For long-running background loops (rare).

Q: Should I wrap async I/O in `Task.Run`? **A:** No, that wastes a thread.

Q: Why is the thread pool more efficient? **A:** Threads are reused, reducing allocation and context switching.

Q40: Task.Run vs. Task.Factory.StartNew

Direct Answer: Use `Task.Run` for everything modern — it has safe defaults and properly handles async lambdas. `Task.Factory.StartNew` is the older, lower-level API. Its main pitfall is that with `async () => ...`, it returns `Task<Task>` instead of `Task`, which silently breaks awaiting. Reach for `StartNew` only when you need advanced options like `TaskCreationOptions.LongRunning`.

- `Task.Run` → modern, safe.
- `StartNew` → old, tricky with async lambdas.
- Use `LongRunning` for dedicated thread.
- Avoid `StartNew` in everyday code.

```
var t = Task.Run(async () => await DoAsync());
// Edge case: dedicated thread
Task.Factory.StartNew(() => MonitorLoop(), TaskCreationOptions.LongRunning);
```

Q: What's the bug in `StartNew(async () => ...)`? **A:** It returns `Task<Task>` — outer completes early.

Q: When to use `LongRunning`? **A:** Background services, file watchers, message consumers.

Q: Are continuations the same? **A:** Both support `ContinueWith`, but Task.Run is simpler.

Q41: Deadlocks — `.Result` vs. `await` (IMPORTANT)

Direct Answer: Calling `.Result` or `.Wait()` on an async method **blocks** the calling thread. In UI apps or classic ASP.NET, this often causes deadlocks because the awaited continuation tries to come back to the same blocked thread. `await` is non-blocking — it releases the thread until the work finishes. ASP.NET Core has no SynchronizationContext, so deadlocks are less likely there, but blocking still wastes pool threads.

- `.Result` / `.Wait()` → blocks thread.
- `await` → non-blocking.
- Mixing sync + async causes deadlocks.
- "Async all the way" is the safe rule.

```
// Bad — may deadlock or starve threads
var data = GetDataAsync().Result;
// Good
var data = await GetDataAsync();
```

Q: Why does `.Result` deadlock in WinForms? **A:** UI thread is blocked, but continuation needs that same thread.

Q: What's "thread pool starvation"? **A:** Too many blocked threads make the app freeze.

Q: Safer alternative when blocking is unavoidable? **A:** `GetAwaiter().GetResult()` — unwraps exceptions cleanly.

Q42: CancellationToken — Stopping Long-Running Work

Direct Answer: `CancellationToken` provides cooperative cancellation for async operations. The caller creates a `CancellationTokenSource`, passes its `Token` to async methods, and calls `Cancel()` when needed. The async code checks `IsCancellationRequested` or calls `ThrowIfCancellationRequested()`. ASP.NET Core gives `HttpContext.RequestAborted` automatically — when the user disconnects, you can stop processing.

- Cooperative — code must check the token.
- Built-in async APIs accept tokens.
- Use linked tokens to combine signals.
- Don't swallow `OperationCanceledException`.

```
public async Task<IActionResult> Get(CancellationToken ct) {
    var data = await _db.Orders.ToListAsync(ct);
    return Ok(data);
}
```

Q: How do you add a timeout? **A:** Use `CancellationTokenSource.CancelAfter(TimeSpan)`.

Q: What if I ignore the token? **A:** Cancellation has no effect — defeats the purpose.

Q: How to combine multiple tokens? **A:** `CreateLinkedTokenSource(token1, token2)`.

Q43: ValueTask vs. Task

Direct Answer: `Task` is a reference type — every async call allocates a heap object. `ValueTask<T>` is a struct designed for hot paths that **often complete synchronously**, like cache lookups. It avoids unnecessary allocation when the result is already available. But it has rules: you can only `await` it once, you can't cache it. Use `Task` by default; switch to `ValueTask` only after profiling.

- Task → heap allocation always.
- ValueTask → struct, less allocation for sync paths.
- Await only once.
- Default to `Task`.

```
public ValueTask<User> GetUserAsync(int id) {
    if (_cache.TryGetValue(id, out var u)) return new ValueTask<User>(u);
    return new ValueTask<User>(LoadAsync(id));
}
```

Q: Default async return type? **A:** `Task` or `Task<T>`.

Q: Can we use Task.WhenAll with ValueTask? **A:** Convert with `.AsTask()` first.

Q: Where does ValueTask shine? **A:** Caches, pooled buffers, fast-path APIs.

Q44: ConfigureAwait(false) — Why It Matters

Direct Answer: `ConfigureAwait(false)` tells `await` not to capture the current SynchronizationContext, so the continuation can run on any thread pool thread. This avoids deadlocks and improves performance in **library code**. In ASP.NET Core there's no SynchronizationContext, so it's

effectively a no-op there — but still recommended in shared libraries that may be consumed by older platforms.

- Skips capturing SyncContext.
- Useful in NuGet/library code.
- Avoids deadlocks in WinForms/WPF.
- No-op in ASP.NET Core.

```
public async Task<string> FetchAsync() {
    using var resp = await _http.SendAsync(req).ConfigureAwait(false);
    return await resp.Content.ReadAsStringAsync().ConfigureAwait(false);
}
```

Q: Should I use it in controllers? **A:** Not necessary — ASP.NET Core has no SyncContext.

Q: When is it required? **A:** WinForms, WPF, libraries consumed by them.

Q: Can it be set globally? **A:** Yes, via `[ConfigureAwait(false)]` at assembly level (C# 11).

Q45: lock vs. Monitor vs. SemaphoreSlim

Direct Answer: `lock` is the simplest way to ensure only one thread enters a code block — it's actually compiled into `Monitor.Enter` and `Monitor.Exit`. `Monitor` provides finer control with timeouts and `TryEnter`. `SemaphoreSlim` allows N threads at once and supports `async` (`WaitAsync`) — use it inside async code where `lock` is forbidden. Always lock on a private object, never on `this` or `string`.

- lock → simple, sync only.
- Monitor → lock with timeout/control.
- SemaphoreSlim → async-friendly, N threads.
- Lock on private readonly object.

```
private readonly object _gate = new();
lock (_gate) { _count++; }

private readonly SemaphoreSlim _sem = new(1,1);
await _sem.WaitAsync();
try { /* critical */ } finally { _sem.Release(); }
```

Q: Why not lock on `this`? **A:** External code may also lock on it, causing deadlocks.

Q: Async-safe alternative to lock? **A:** `SemaphoreSlim(1, 1)`.

Q: Lock-free counter? **A:** `Interlocked.Increment(ref count)`.

Q46: Task.WhenAll vs. Task.WhenAny

Direct Answer: `Task.WhenAll` waits for **all tasks** to finish (or any to throw). `Task.WhenAny` completes when the **first task** finishes. Use `WhenAll` for parallel fan-out — like calling 3 APIs at once. Use `WhenAny` for racing patterns like "first one wins" or implementing timeouts using `Task.Delay`. Both are essential for writing parallel async code.

- WhenAll → wait for everything.
- WhenAny → first to finish wins.
- Useful for parallel API calls.
- Used to implement timeouts.

```
var results = await Task.WhenAll(api1, api2, api3);

var work = DoLongAsync();
var winner = await Task.WhenAny(work, Task.Delay(5000));
if (winner ≠ work) throw new TimeoutException();
```

Q: Does WhenAll throw if one task fails? **A:** Yes, but only the first exception is rethrown.

Q: How to throttle WhenAll? **A:** Use `Parallel.ForEachAsync` or `SemaphoreSlim`.

Q: Difference between WaitAll and WhenAll? **A:** WaitAll blocks; WhenAll is awaitable.

Q47: Async Streams — IAsyncEnumerable (MODERN)

Direct Answer: `IAsyncEnumerable<T>` (C# 8) lets you stream data asynchronously, item by item, using `await foreach`. It's perfect for paged APIs, large DB results, or live data feeds. Combined with `yield return` in async methods, you don't need to load everything into memory. EF Core, ASP.NET Core (gRPC, server-sent events), and channels all support it natively.

- Async + lazy iteration.
- Use `await foreach` to consume.
- Great for streaming large data.
- Use `[EnumeratorCancellation]` for token support.

```
public async IAsyncEnumerable<Order> StreamAsync(
    [EnumeratorCancellation] CancellationToken ct = default) {
    await foreach (var o in _db.Orders.AsAsyncEnumerable().WithCancellation(ct))
        yield return o;
}
```

Q: Difference from `Task<List<T>>`? **A:** `Task<List<T>>` loads everything; async enumerable streams.

Q: How to cancel iteration? **A:** Use `WithCancellation(token)`.

Q: Does EF Core support it? **A:** Yes — `AsAsyncEnumerable()` enables streaming.

CATEGORY 5: ASP.NET CORE & .NET CORE

Q48: .NET Framework vs .NET Core vs .NET 5+ (IMPORTANT)

.NET Framework is the original Windows-only platform (last version 4.8). .NET Core (1–3.1) is the cross-platform, open-source rewrite — runs on Windows, Linux, macOS. From .NET 5 onward, Microsoft unified everything into a single platform called just ".NET" (5, 6, 7, 8, 9). All new development should target the latest .NET 8 LTS or newer; legacy apps stay on Framework.

- .NET Framework → Windows only, legacy.
- .NET Core → cross-platform, lightweight.
- .NET 5+ → unified modern platform.
- .NET 8 is the current LTS.

```
.NET Framework 4.8  → Legacy desktop / WebForms
.NET Core 3.1       → Cross-platform LTS
.NET 8 / 9          → Modern unified platform
```

Q: Should I start a new project on .NET Framework? **A:** No, always use .NET 8+.

Q: What is .NET Standard? **A:** A specification for sharing libraries across .NET versions.

Q: What's an LTS release? **A:** Long-Term Support — supported for 3 years.

Q49: Dependency Injection — Transient, Scoped, Singleton (IMPORTANT)

ASP.NET Core has built-in DI with three lifetimes:

- **Transient** — new instance every time it's requested.
- **Scoped** — one instance per HTTP request.
- **Singleton** — one instance for the entire app lifetime. Use `Scoped` for `DbContext` and repositories, `Singleton` for stateless utilities, and `Transient` for lightweight services like validators.
- Transient → new every time.
- Scoped → per request.
- Singleton → one for app lifetime.
- Avoid injecting Scoped into Singleton.

```
services.AddScoped<IOrderRepository, OrderRepository>();
services.AddSingleton<IClock, SystemClock>();
services.AddTransient<IEmailValidator, EmailValidator>();
```

Q: What is captive dependency? **A:** Holding a Scoped/Transient inside a Singleton — causes bugs.

Q: For HTTP calls in singletons? **A:** Use `IHttpClientFactory`.

Q: Why is DbContext Scoped? **A:** It's not thread-safe and tracks entities per request.

Q50: Middleware Pipeline in ASP.NET Core (IMPORTANT)

Direct Answer: Middleware are components that handle each HTTP request in a pipeline. Each one can do work before and after calling `next()`, like a Russian-doll structure. They handle cross-cutting concerns like authentication, routing, logging, and exception handling. The order in `Program.cs` is critical — `UseAuthentication` must come before `UseAuthorization`.

- Form a request-response pipeline.
- Order matters.
- Can short-circuit by not calling next.
- Custom middleware = class with `InvokeAsync(HttpContext)`.

```
app.UseExceptionHandler("/error");
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

Q: What if `UseAuthorization` is before `UseAuthentication`? **A:** Authorization always denies — auth never sets the user.

Q: How do I write custom middleware? **A:** Create a class with `InvokeAsync(HttpContext)` and `RequestDelegate`.

Q: Difference between middleware and filters? **A:** Middleware = pipeline-level; filters = MVC-action-level.

Q51: Filters — Action, Result, Exception, Authorization

Filters in ASP.NET Core run within MVC action execution and let you inject logic at specific points:

- **Authorization filter** → first, controls access.
- **Action filter** → before/after action execution.
- **Result filter** → before/after the response.
- **Exception filter** → handles unhandled errors. They're great for cross-cutting concerns like logging, validation, or auditing — without touching every controller.

- Run inside MVC pipeline.
- Apply globally, per controller, or per action.
- Implement via attributes or interfaces.
- Use middleware for non-MVC concerns.

```
public class LogActionFilter : IActionFilter {
    public void OnActionExecuting(ActionExecutingContext c) { /* before */ }
    public void OnActionExecuted(ActionExecutedContext c) { /* after */ }
}
```

Q: Filter vs. middleware? **A:** Filters know about MVC (action, model); middleware doesn't.

Q: How to apply globally? **A:** Add to `MvcOptions.Filters` in Program.cs.

Q: Order of filter execution? **A:** Authorization → Resource → Action → Result → Exception.

Q52: Garbage Collection — Generations 0, 1, 2

Direct Answer: .NET GC is **generational** — it groups objects by age:

- **Gen 0** → newly created, collected most often (cheap).
- **Gen 1** → survived one Gen 0 collection.
- **Gen 2** → long-lived objects, collected rarely (expensive). Large objects (>85 KB) go to the **LOH (Large Object Heap)** and are collected with Gen 2. Goal: most objects die young, so Gen 0 cleanup is fast.
- Gen 0/1/2 → short to long-lived.
- LOH for objects >85 KB.
- Gen 2 collections are slow.
- Don't call `GC.Collect()` manually.

```
var pooled = ArrayPool<byte>.Shared.Rent(1024);  // reuse, less GC pressure
```

Q: Should I call `GC.Collect()`? **A:** Almost never — the GC manages it better.

Q: How to reduce GC pressure? **A:** Allocate less, use pooling, reuse buffers.

Q: What is Server GC? **A:** A multi-threaded GC tuned for ASP.NET Core servers.

Q53: IHttpClientFactory — Why Not new HttpClient()?

Direct Answer: Creating a new `HttpClient` per request leaks sockets (TIME_WAIT). Storing one as a static field avoids that but can use stale DNS. `IHttpClientFactory` solves both — it pools and rotates handlers, manages DNS lifetimes, and lets you configure named or typed clients with policies (Polly retries, timeouts). Always use it in ASP.NET Core, never raw `new HttpClient()`.

- Avoids socket exhaustion.
- Refreshes DNS automatically.
- Supports named/typed clients.
- Integrates with Polly for retries.

```
services.AddHttpClient("api", c => c.BaseAddress = new Uri("https://api.com/"));
var client = httpFactory.CreateClient("api");
```

Q: Why not use static HttpClient? **A:** DNS doesn't refresh — stale endpoints can cause failures.

Q: What is a typed client? **A:** A class with HttpClient injected via constructor.

Q: How to add retries? **A:** Use `AddPolicyHandler` with Polly.

Q54: IOptions Pattern — Strongly Typed Config

Direct Answer: The `IOptions<T>` pattern binds sections of `appsettings.json` to strongly typed C# classes. You register them in DI and inject `IOptions<MySettings>`. There's also `IOptionsSnapshot<T>` (per request, supports reload) and `IOptionsMonitor<T>` (singleton with change notifications). This avoids hard-coded strings and gives type safety for configuration.

- Binds JSON config to classes.
- Register with `services.Configure<T>(...)`.
- IOptionsSnapshot → per request, reloads.
- IOptionsMonitor → live reload, singleton.

```
services.Configure<EmailSettings>(config.GetSection("Email"));

public class EmailService {
    public EmailService(IOptions<EmailSettings> opts) { ... }
}
```

Q: Difference IOptions vs. IOptionsSnapshot? **A:** Snapshot reloads per request; Options is fixed at startup.

Q: When to use IOptionsMonitor? **A:** Singletons that need live reload notifications.

Q: Can I validate config? **A:** Yes, with `ValidateDataAnnotations()` or `ValidateOnStart()`.

Q55: appsettings.json and Configuration Hierarchy

Direct Answer: ASP.NET Core reads configuration from multiple sources in order: `appsettings.json`, `appsettings.{Environment}.json`, environment variables, command-line args, and user secrets. Later sources override earlier ones. So `appsettings.Development.json` overrides production settings during development. This layered design lets you keep secrets out of source code and adapt configuration per environment.

- Layered config — last source wins.
- Environment-based overrides.

- User secrets for dev secrets.
- Environment variables for prod secrets.

```
// appsettings.json
{ "ConnectionStrings": { "Default": "Server=..." } }
```

```
var conn = builder.Configuration.GetConnectionString("Default");
```

Q: Where do you store secrets? **A:** User secrets (dev), Azure Key Vault or env vars (prod).

Q: How to get current environment? **A:** `IWebHostEnvironment.EnvironmentName` or `ASPNETCORE_ENVIRONMENT`.

Q: Why use IConfiguration? **A:** Unified API across all config sources.

Q56: Hosted Services and BackgroundService

Direct Answer: A **Hosted Service** runs background tasks within an ASP.NET Core app — like queue listeners, schedulers, or recurring jobs. Implement `IHostedService` (with `StartAsync`/`StopAsync`) or inherit from `BackgroundService` and override `ExecuteAsync`. They're registered with `services.AddHostedService<T>()` and run for the lifetime of the application.

- Run alongside the web app.
- Use `BackgroundService` for long-running loops.
- Register via `AddHostedService<T>`.
- Respect `CancellationToken` for graceful shutdown.

```
public class CleanupService : BackgroundService {
    protected override async Task ExecuteAsync(CancellationToken ct) {
        while (!ct.IsCancellationRequested) {
            await DoCleanup();
            await Task.Delay(TimeSpan.FromMinutes(5), ct);
        }
    }
}
```

Q: When should I use BackgroundService? **A:** Periodic jobs, queue consumers, monitors.

Q: Difference: BackgroundService vs. Timer? **A:** BackgroundService respects DI and graceful shutdown.

Q: How to inject scoped services? **A:** Use `IServiceScopeFactory` and create a scope inside the loop.

Q57: Logging with ILogger

Direct Answer: ASP.NET Core has built-in logging via `ILogger<T>` that supports multiple providers (Console, Debug, File, Application Insights, Serilog, etc.). It uses **structured logging** — placeholders like `{UserId}` are recorded as fields, not just text. Configure log levels in `appsettings.json` per category. Avoid string concatenation; let placeholders preserve structure.

- Use `ILogger<T>` (T = class name).
- Structured logging with `{Placeholders}`.
- Log levels: Trace, Debug, Information, Warning, Error, Critical.
- Configure providers via DI.

```
public class UserService {
    private readonly ILogger<UserService> _log;
    public UserService(ILogger<UserService> log) => _log = log;
    public void Greet(int id) => _log.LogInformation("Hello {UserId}", id);
}
```

Q: Why use placeholders, not interpolation? **A:** Preserves structured fields for searchable logs.

Q: What's a popular third-party logger? **A:** Serilog with sinks for files, Seq, Elastic.

Q: What is `ILogger<T>` benefit? **A:** Auto-categorized by class name for filtering.

Q58: Minimal APIs vs. Controllers (MODERN)

Direct Answer: Minimal APIs (introduced in .NET 6) let you define endpoints directly in `Program.cs` without controllers. They're concise, fast, and great for microservices, simple APIs, or quick prototypes. Controllers are still preferred for large apps with many endpoints, complex routing, filters, and conventions. Both share routing, DI, and middleware — choose based on team size and complexity.

- Minimal APIs → less code, fast.
- Controllers → structured, conventions-based.
- Both share routing & DI.
- Use Minimal APIs for small/microservices.

```
app.MapGet("/users/{id}", async (int id, IUserService svc) ⇒
    Results.Ok(await svc.GetAsync(id)));
```

Q: Can Minimal APIs use filters? **A:** Yes, since .NET 7 with endpoint filters.

Q: Are Minimal APIs faster? **A:** Slightly — fewer abstractions, but mostly comparable.

Q: Can I mix both? **A:** Yes, in the same project.

Q59: Authentication vs. Authorization (IMPORTANT)

Direct Answer: **Authentication** is verifying *who* the user is (login, JWT, OAuth, cookies). **Authorization** is deciding *what* they can do (roles, policies, claims). They run in that order in the middleware pipeline. ASP.NET Core supports JWT, cookies, OAuth2, OpenID Connect, and policy-based authorization out of the box.

- Authentication = identity check.
- Authorization = permission check.
- Auth middleware runs before authz.
- Policy-based authz is most flexible.

```
app.UseAuthentication();
app.UseAuthorization();

[Authorize(Roles = "Admin")]
public IActionResult AdminOnly() ⇒ Ok();
```

Q: Difference between roles and claims? **A:** Roles are a kind of claim; claims are key-value identity facts.

Q: What is a policy? **A:** A reusable authorization rule made of claim/role requirements.

Q: Best auth scheme for SPAs? **A:** JWT bearer or OpenID Connect (OAuth2).

Q60: JWT Tokens in ASP.NET Core (MODERN)

Direct Answer: JWT (JSON Web Token) is a stateless, signed token used for authentication in APIs. It contains a header, payload (claims), and signature. The client gets a JWT after login and sends it in every request via `Authorization: Bearer <token>`. The server validates the signature (no DB lookup

needed). JWTs are great for distributed APIs but can't be revoked easily — use short expiry + refresh tokens.

- Stateless, signed token.
- Header.Payload.Signature (Base64).
- Sent via `Authorization: Bearer ...`.
- Use short expiry + refresh tokens.

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(opts => {
        opts.TokenValidationParameters = new() {
            ValidateIssuer = true, ValidateAudience = true,
            ValidateLifetime = true, ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(key)
        };
    });
```

Q: Can JWT be revoked? **A:** Not easily — use a blacklist or short expiry + refresh.

Q: Where to store JWT in browser? **A:** HttpOnly cookie for safety; avoid localStorage.

Q: Difference: JWT vs. session cookie? **A:** JWT = stateless, cookie = stateful (server stores session).

CATEGORY 6: ENTITY FRAMEWORK CORE

Q61: EF Core vs. ADO.NET vs. Dapper

Direct Answer: **ADO.NET** is the lowest-level data access — you write raw SQL and manually map results. **Dapper** is a micro-ORM on top of ADO.NET — fast, simple, but no change tracking. **EF Core** is a full ORM with LINQ, change tracking, migrations, and lazy loading. Use EF Core for complex domains, Dapper for high-performance read-heavy paths, and ADO.NET only for low-level needs.

- ADO.NET → raw SQL, manual mapping.
- Dapper → fast micro-ORM.
- EF Core → full ORM with LINQ + tracking.
- Choose based on complexity vs performance.

```
// EF Core
var orders = await _ctx.Orders.Where(o => o.Status == "Open").ToListAsync();
// Dapper
var orders = await _conn.QueryAsync<Order>("SELECT * FROM Orders WHERE Status='Open'");
```

Q: Which is fastest? **A:** Dapper, slightly ahead of EF Core for raw SELECTs.

Q: Can I mix EF Core + Dapper? **A:** Yes — EF for writes, Dapper for hot read paths.

Q: Why do teams pick EF Core? **A:** Productivity — LINQ, migrations, change tracking.

Q62: Code First vs. Database First (IMPORTANT)

Direct Answer: **Code First** means you define your model classes first, and EF Core generates the database via migrations. **Database First** starts from an existing database and scaffolds entity classes (`dotnet ef dbcontext scaffold`). Code First is preferred for new projects because the model is version-controlled with the code. Database First is useful for legacy databases.

- Code First → classes first, migrations create DB.
- Database First → scaffold from DB.
- Code First is more popular today.
- Use Database First for legacy systems.

```
# Code First
dotnet ef migrations add InitialCreate
dotnet ef database update

# Database First
dotnet ef dbcontext scaffold "ConnString" Microsoft.EntityFrameworkCore.SqlServer
```

Q: What is Model First? **A:** Older approach using a designer file (.edmx) — rarely used now.

Q: Can we mix the two? **A:** Yes, by scaffolding then customizing.

Q: Why is Code First preferred? **A:** Source control, migrations, fluent API control.

Q63: EF Core Migrations

Direct Answer: Migrations track changes to your model and apply them to the database in versioned steps. Use `dotnet ef migrations add MigrationName` to create one and

`dotnet ef database update` to apply it. Each migration produces an `Up` (apply) and `Down` (revert) method. In production, run migrations as part of deployment, not on app startup.

- Versioned schema changes.
- Add → Update workflow.
- Up/Down methods for revert.
- Apply during deployment, not at runtime.

```
dotnet ef migrations add AddOrderTotal
dotnet ef database update
```

Q: How to revert a migration? **A:** `dotnet ef database update PreviousMigrationName`.

Q: Can I edit migration files? **A:** Yes, before running update — useful for data fixes.

Q: What is `__EFMigrationsHistory`? **A:** A table tracking applied migrations.

Q64: AsNoTracking vs. Tracking (TRICKY)

Direct Answer: By default, EF Core **tracks** entities you query — it watches for changes so `SaveChanges` knows what to update. This adds memory and CPU overhead. For read-only queries, call `AsNoTracking()` to skip tracking — typically 30–50% faster on read paths and reduces memory. Always use it in API endpoints that just return data.

- Tracked by default.
- `AsNoTracking()` → skip tracking, faster reads.
- Use for read-only API responses.
- Saves memory in large queries.

```
var users = await _ctx.Users.AsNoTracking().ToListAsync();
```

Q: When should I keep tracking? **A:** When loading data to update or delete.

Q: Can I make it default? **A:** Yes, with `ChangeTracker.QueryTrackingBehavior = NoTracking`.

Q: What is `AsNoTrackingWithIdentityResolution`? **A:** Like AsNoTracking but de-duplicates entities in results.

Q65: Lazy vs. Eager vs. Explicit Loading

- **Eager loading** uses `Include()` to load related data in one query upfront.
- **Lazy loading** loads related data only when accessed (needs proxies enabled).
- **Explicit loading** lets you call `Load()` manually after the main query. Eager loading is the safest default — it avoids the **N+1 problem** where accessing each child triggers a separate SQL query.
- Eager → `Include()`, one query.
- Lazy → automatic, on access.
- Explicit → manual control.
- Avoid lazy in APIs — causes N+1.

```
var orders = await _ctx.Orders
    .Include(o ⇒ o.Items)
    .ToListAsync();
```

Q: What is the N+1 problem? **A:** One query for parents + N queries for children — kills performance.

Q: How to enable lazy loading? **A:** Add `Microsoft.EntityFrameworkCore.Proxies` and `UseLazyLoadingProxies()`.

Q: Default in EF Core? **A:** Lazy loading is OFF by default.

Q66: Include vs. ThenInclude

Direct Answer: `Include` loads a directly related entity. `ThenInclude` loads a nested related entity from the previously included one. Together they let you navigate relationships and load deep object graphs in a single query — avoiding N+1. Be careful — over-including data can lead to cartesian explosion (huge result sets).

- Include → first level relationship.
- ThenInclude → nested relationship.
- One SQL query, multiple joins.
- Avoid loading too much.

```
var orders = await _ctx.Orders
    .Include(o => o.Items)
        .ThenInclude(i => i.Product)
    .ToListAsync();
```

Q: Cartesian explosion? **A:** Multiple Includes can multiply rows — use split queries.

Q: What is `AsSplitQuery()`? **A:** Splits Include into multiple SQL queries to avoid duplication.

Q: Can I filter Include? **A:** Yes — `.Include(o => o.Items.Where(...))`.

Q67: Transactions in EF Core

Direct Answer: EF Core wraps each `SaveChanges` call in an implicit transaction. For multi-step operations across multiple saves, use `Database.BeginTransactionAsync()` to manage them explicitly. You commit on success and roll back on exception. For distributed transactions across multiple databases, use `TransactionScope` (with caution — it's heavier).

- `SaveChanges` is auto-transactional.
- Use explicit transactions for multi-step ops.
- Rollback on exception.
- TransactionScope for distributed cases.

```
using var tx = await _ctx.Database.BeginTransactionAsync();
try {
    await DoWork1();
    await DoWork2();
    await tx.CommitAsync();
} catch { await tx.RollbackAsync(); throw; }
```

Q: Default isolation level? **A:** Read Committed (configurable).

Q: Why use explicit transactions? **A:** When you need atomicity across multiple `SaveChanges`.

Q: Are nested transactions allowed? **A:** EF Core uses savepoints, not true nesting.

Q68: Stored Procedures with EF Core

Direct Answer: EF Core supports stored procedures via `FromSqlRaw` (queries) and `ExecuteSqlRaw` (commands). EF 7+ also lets you map insert/update/delete operations to stored procs via fluent API.

SPs are useful for performance-critical logic, complex aggregations, or when DBAs maintain logic in SQL. Always parameterize inputs to avoid SQL injection.

- `FromSqlRaw` / `FromSqlInterpolated` for queries.
- `ExecuteSqlRaw` for non-queries.
- EF 7+ maps CRUD to SPs via fluent API.
- Always use parameters.

```
var users = await _ctx.Users
    .FromSqlInterpolated($"EXEC GetActiveUsers {minAge}")
    .ToListAsync();
```

Q: Why use `FromSqlInterpolated`? **A:** Auto-parameterizes interpolated strings — safe from injection.

Q: Can I call SPs returning multiple result sets? **A:** Use raw ADO.NET or specific EF extensions.

Q: Are SPs always faster? **A:** Not necessarily — modern EF generates very good SQL.

CATEGORY 7: MODERN .NET & ADVANCED CONCEPTS

Q69: Reflection — What It Is and Why It's Slow

Direct Answer: Reflection lets you inspect and invoke types at runtime — read attributes, list properties, create instances, call methods by name. It's used by serializers, ORMs, DI containers, and validators. Reflection is slow because it skips compile-time checks and goes through metadata lookups, security checks, and boxing. For hot paths, cache the reflection results or use **source generators** (zero runtime cost).

- Inspect/invoke types at runtime.
- Used by serializers, EF, DI.
- 10–100× slower than direct calls.
- Cache results or use source generators.

```
var prop = typeof(User).GetProperty("Name");
var name = prop?.GetValue(user);
```

Q: Where is reflection used in .NET? **A:** Serializers, DI containers, MVC routing, EF Core.

Q: Faster alternative? **A:** Source generators or compiled expressions (`Expression.Compile`).

Q: Is reflection AOT-friendly? **A:** Limited — many reflection APIs aren't supported in trimming/AOT.

Q70: Delegates vs. Events

Direct Answer: A **delegate** is a type-safe pointer to a method. An **event** is a delegate field with restricted access — outside the class, only `+=` and `-=` are allowed; you can't invoke or reassign it. Events implement the publish/subscribe pattern in C#. Forgetting to unsubscribe (`-=`) is a common cause of memory leaks.

- Delegate = method reference.
- Event = restricted delegate (pub/sub).
- `+=` to subscribe, `-=` to unsubscribe.
- Forgotten `-=` causes memory leaks.

```
public event EventHandler<OrderEventArgs>? OrderShipped;
OrderShipped?.Invoke(this, new OrderEventArgs(...));
```

Q: Difference: Action vs. Func? **A:** Action returns void; Func returns a value.

Q: Can multiple handlers attach? **A:** Yes, delegates are multicast.

Q: Modern alternatives to events? **A:** `IObservable<T>`, `Channel<T>`, message buses (MediatR).

Q71: Func vs. Action vs. Predicate (TRICKY)

Direct Answer: `Action<T>` represents a method returning **void** with up to 16 parameters. `Func<T, TResult>` represents a method that **returns a value**. `Predicate<T>` is a specialized `Func<T, bool>` — used for tests like `List.Find`. They're built-in delegate types so you don't have to declare your own. Use `Func` and `Action` for most cases.

- Action → returns void.
- Func → returns a value.
- Predicate → bool, takes one input.
- Avoid declaring custom delegates when these fit.

```
Action<string> log = Console.WriteLine;
Func<int, int, int> add = (a, b) => a + b;
Predicate<int> isEven = n => n % 2 == 0;
```

Q: Why prefer Func over Predicate? **A:** Func is more general; LINQ uses `Func<T,bool>`.

Q: Max parameters in Func/Action? **A:** 16.

Q: Are these reference types? **A:** Yes — delegates are reference types.

Q72: Custom Attributes

Direct Answer: Custom attributes attach metadata to classes, methods, properties, etc. Inherit from `System.Attribute` and use `[AttributeUsage(...)]` to control where it can be applied. Read attributes at runtime via reflection. Common examples: `[HttpGet]`, `[Required]`, `[Authorize]`, `[Table]`. They're widely used by frameworks for validation, routing, and ORM mapping.

- Inherit from `Attribute`.
- Use `[AttributeUsage]` to limit targets.
- Read with reflection.
- Cache reflection results in hot paths.

```
[AttributeUsage(AttributeTargets.Method)]
public class FeatureFlagAttribute : Attribute {
    public string Name { get; }
    public FeatureFlagAttribute(string name) => Name = name;
}
```

Q: How to read an attribute? **A:** `member.GetCustomAttribute<T>()`.

Q: Are attributes runtime or compile-time? **A:** Stored at compile-time, read at runtime.

Q: Performance concern? **A:** Reflection is slow — cache lookups for hot paths.

Q73: Exception Handling — throw vs. throw ex (TRICKY)

Direct Answer: `throw ex;` resets the **stack trace** to the rethrow point, hiding where the original exception came from — making debugging painful. `throw;` keeps the original stack trace intact. If

you want to add context, wrap it: `throw new MyException("...", ex)` so the original is preserved as `InnerException`. Never catch `Exception` and silently swallow it.

- `throw;` → keeps stack trace.
- `throw ex;` → resets trace (bad).
- Wrap with InnerException for context.
- Catch specific exceptions when possible.

```
try { Save(); }
catch (DbUpdateException ex) {
    _log.LogError(ex, "DB error");
    throw; // preserves trace
}
```

Q: Should we ever catch `Exception`? **A:** Only at the very top, e.g., global handler.

Q: What does `when` filter do? **A:** Conditional catch without entering and re-throwing.

Q: What is `ExceptionDispatchInfo.Capture`? **A:** Preserves the trace when re-throwing across async boundaries.

Q74: Records vs. Class vs. Struct (MODERN)

Direct Answer: A **record** (C# 9) is a reference type built for immutable data with **value-based equality** — two records are equal if all their properties are equal. A **class** uses reference equality and is mutable by default. A **struct** is a value type — copied on assignment, no inheritance. Use records for DTOs, classes for entities and services, and structs for small value-like types.

- Record → reference type, value equality.
- Class → reference type, identity equality.
- Struct → value type, copied.
- Use records for DTOs and value objects.

```
public record OrderDto(int Id, decimal Total);
var a = new OrderDto(1, 99);
var b = new OrderDto(1, 99);
Console.WriteLine(a == b); // True (value equality)
```

Q: Should I use records for EF entities? **A:** No — value equality conflicts with identity tracking.

Q: Difference: record vs. record struct? **A:** record = reference type; record struct = value type.

Q: What is `with` expression? **A:** Non-destructive copy that changes only specified properties.

Q75: Top-Level Statements (MODERN)

Direct Answer: Top-level statements (C# 9+) let you write a `Program.cs` without explicitly declaring `Main()` or a class. The compiler generates them behind the scenes. This makes minimal apps and learning examples cleaner. Modern .NET templates use this style by default. You can still add classes, methods, and using directives in the same file.

- No `Main()` boilerplate.
- Compiler auto-generates Program class.
- Cleaner, less ceremony.
- Default in .NET 6+ templates.

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
app.MapGet("/", () => "Hello!");
app.Run();
```

Q: Can I still write Main()? **A:** Yes, but only one file can have top-level statements.

Q: Are async top-level statements allowed? **A:** Yes — `await` works at the top level.

Q: Where can I declare types? **A:** Below the statements, in the same file.

Q76: Global Usings and File-Scoped Namespaces (MODERN)

Direct Answer: **Global usings** (C# 10) let you add `using` directives once that apply across the entire project — no more repeating `using System;` everywhere. **File-scoped namespaces** also reduce boilerplate by letting you write `namespace MyApp;` once at the top instead of wrapping the whole file. Both make code cleaner and reduce indentation.

- `global using` → applies project-wide.
- File-scoped namespace → no braces.
- Reduces boilerplate.
- Default in .NET 6+ project templates.

```
// GlobalUsings.cs
global using System;
global using System.Linq;

// File-scoped namespace
namespace MyApp.Services;
public class UserService { }
```

Q: Where should global usings live? **A:** A dedicated `GlobalUsings.cs` or csproj `<ItemGroup>`.

Q: Pros of file-scoped namespace? **A:** Less indentation, cleaner code.

Q: Can both old and new styles coexist? **A:** Yes, but stick to one for consistency.

Q77: Span and Memory (MODERN)

Direct Answer: `Span<T>` is a stack-only struct that gives you a typed view over contiguous memory — array, stack-allocated buffer, or unmanaged block — without copying. `Memory<T>` is similar but heap-friendly and safe for async methods. Both are used in performance-critical scenarios like parsing, slicing, and high-throughput I/O. They reduce allocations dramatically.

- Span → stack-only, fast.
- Memory → heap-friendly, async-safe.
- Slicing without copying.
- Used in modern parsers and System.IO.Pipelines.

```
ReadOnlySpan<char> s = "Hello, World".AsSpan().Slice(0, 5);
Console.WriteLine(s.ToString());  // Hello
```

Q: Why can't Span be in an async method? **A:** It's a `ref struct` — must stay on the stack.

Q: Use case for Memory? **A:** Async methods that pass buffers (HTTP, streams).

Q: What is `ArrayPool<T>`? **A:** Reusable pool of arrays — works great with Span.

Q78: Source Generators (MODERN)

Direct Answer: Source generators (C# 9+) are compile-time tools that generate additional C# code based on your existing code. They run during build, see your types, and produce extra source files that the compiler includes. They're the modern replacement for runtime reflection — used by

`System.Text.Json`, regex, logging, MediatR, and DI containers. Result: no runtime cost and AOT-friendly.

- Generate code at compile time.
- Replace runtime reflection.
- AOT/trimming friendly.
- Used by JSON serializer, regex, logging.

```
[JsonSerializable(typeof(User))]
public partial class AppJsonContext : JsonSerializerContext { }
// generated code handles serialization without reflection
```

Q: Why are they faster than reflection? **A:** Generated code runs as plain compiled C#.

Q: Where do they fit in build? **A:** During Roslyn compilation — extra files generated.

Q: Common real-world examples? **A:** `LoggerMessage`, `JsonSerializerContext`, `RegexGenerator`.

APPENDIX

Quick Revision Cheat Sheet

Concept	One-Liner
Stack vs. Heap	Value types store data; reference types store pointers
Boxing	Value type → object on heap (allocation + copy)
const	Compile-time constant, baked into IL
readonly	Runtime, set in constructor
Interface vs. Abstract	Contract vs. partial template
IEnumerable vs. IQueryable	In-memory vs. database
async/await	Non-blocking I/O — no new threads
Task.Run	CPU-bound work only
ConfigureAwait(false)	Library code only
ValueTask	Hot-path, often-sync results
DI Lifetimes	Singleton / Scoped / Transient
throw vs. throw ex	Preserve trace vs. reset trace
record	Value-based equality + with expression
AsNoTracking	Faster read-only EF queries
JWT	Stateless signed token for APIs
Minimal API	Lightweight endpoints in Program.cs
Source Generators	Compile-time code generation

Interview Tips

- **Lead with a one-line crisp answer.** Then expand. Interviewers grade clarity first.
- **Use real examples.** Even small code snippets make answers concrete.

- **Show trade-offs.** "I'd use X because Y, but switch to Z when..."
- **Name common traps.** "captive dependency", "throw ex resets trace", "N+1 problem" — proves real experience.
- **Stay current.** Mention `record`, `IAsyncEnumerable`, `Span<T>`, source generators, Minimal APIs.
- **Don't bluff.** Say "I haven't used that directly, but I expect it works like..."
- **Ask for context.** If a question is vague, clarify — it shows maturity.